EECS 3311 Software Design
Deliverable 2

Task 1

A. Justification for Choosing the Six Design Patterns

The design of the YorkU Lab Equipment Reservation Platform incorporates six object-oriented design patterns. Each pattern was selected because it directly supports one or more system requirements (Req1–Req10) and contributes to a cleaner, more maintainable architecture. The following sections explain the rationale for each pattern.

1. Factory Method Pattern(UserFactory)

The Factory Method pattern is used to centralize and encapsulate the creation of different user types in the system. Instead of instantiating subclasses such as Student, Faculty, Researcher, or Guest directly throughout the application, the UserFactory provides a single creation method that returns the appropriate User subclass based on a type parameter. This improves maintainability by isolating object-creation logic in one place and supports scalability if new user types are added in the future. The factory ensures that all user objects are created consistently and that the calling code remains decoupled from the concrete subclasses, depending only on the abstract User type.

2. Prototype Pattern(Equipment)

The Prototype pattern is applied in the Equipment class through its implementation of Cloneable and its overridden clone() method. This allows the system to create new equipment objects by copying existing ones rather than constructing them from scratch. Cloning is useful when equipment items share similar configurations or when templates are used to generate new instances quickly. The Prototype pattern reduces object-creation complexity and ensures that duplicated equipment inherits the same initial attributes (ID, description, location) while remaining independent objects. This approach is efficient and flexible, especially in environments where equipment types are frequently replicated.

3. Singleton Pattern(UserRegistry)

The UserRegistry class implements the Singleton pattern to guarantee that the system maintains exactly one authoritative registry of all users. By providing a private constructor and a static getInstance() method, the class ensures that all components of the system access the same shared registry. This prevents inconsistencies such as duplicate user lists or conflicting

registration states. Centralizing user storage in a Singleton also simplifies coordination with other system components, such as the HeadLabCoordinator, which relies on the registry for email validation and user lookup. The Singleton pattern is appropriate here because the registry represents a global resource that must remain unique across the entire application.

4. Observer Pattern(Sensor, Observer, EquipmentMonitor)

The Observer pattern is used to support real-time monitoring of equipment usage. The Sensor class acts as the Subject, maintaining a list of observers and notifying them whenever a status change is detected. Observers implement the Observer interface, which defines a uniform update() method. EquipmentMonitor is a concrete observer that reacts to sensor notifications by printing status updates. This pattern decouples the monitoring mechanism from the equipment and sensor logic, allowing multiple observers to be added or removed dynamically without modifying the sensor.

5. Facade Pattern(BookingFacade)

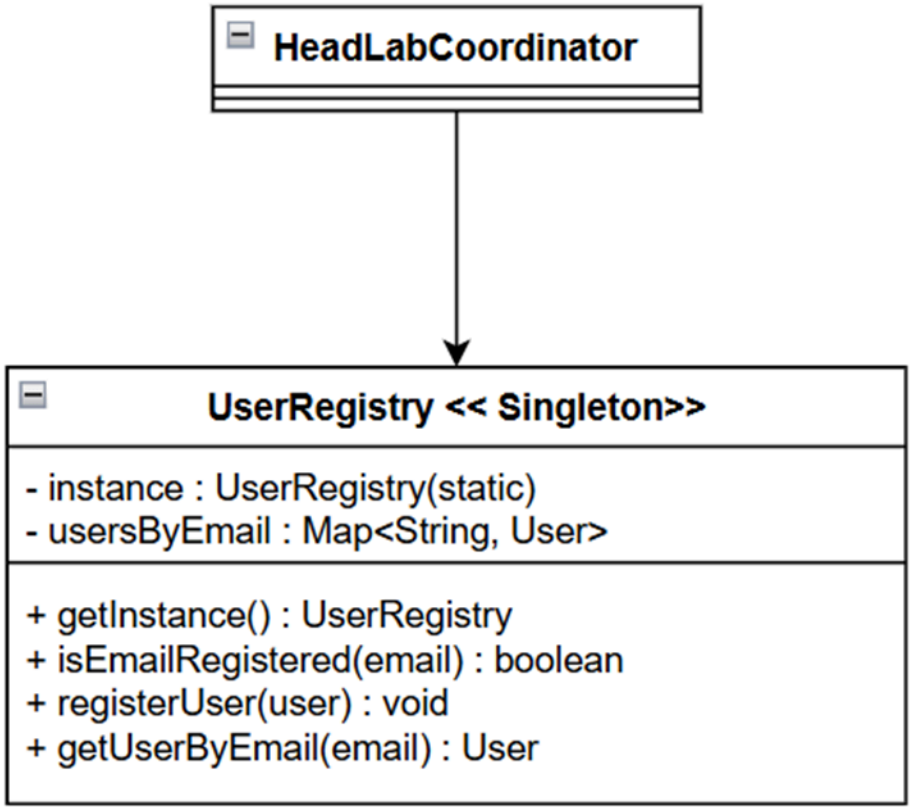
The BookingFacade class provides a simplified interface for managing equipment reservations, cancellations, and extensions. Internally, it coordinates multiple responsibilities: interacting with BookingService, updating equipment availability, calculating costs, and maintaining reservation records. Instead of exposing these subsystems directly to the GUI or command classes, the Facade consolidates them into clear, high-level methods such as reserveEquipment(), cancelReservation(), and extendReservation(). This reduces coupling between the user interface and the underlying booking logic, making the system easier to maintain and modify. The Facade pattern is appropriate here because it hides complexity and presents a clean, unified API for reservation operations.

6. Command Pattern(ReserveCommand, CancelCommand, ExtendCommand)

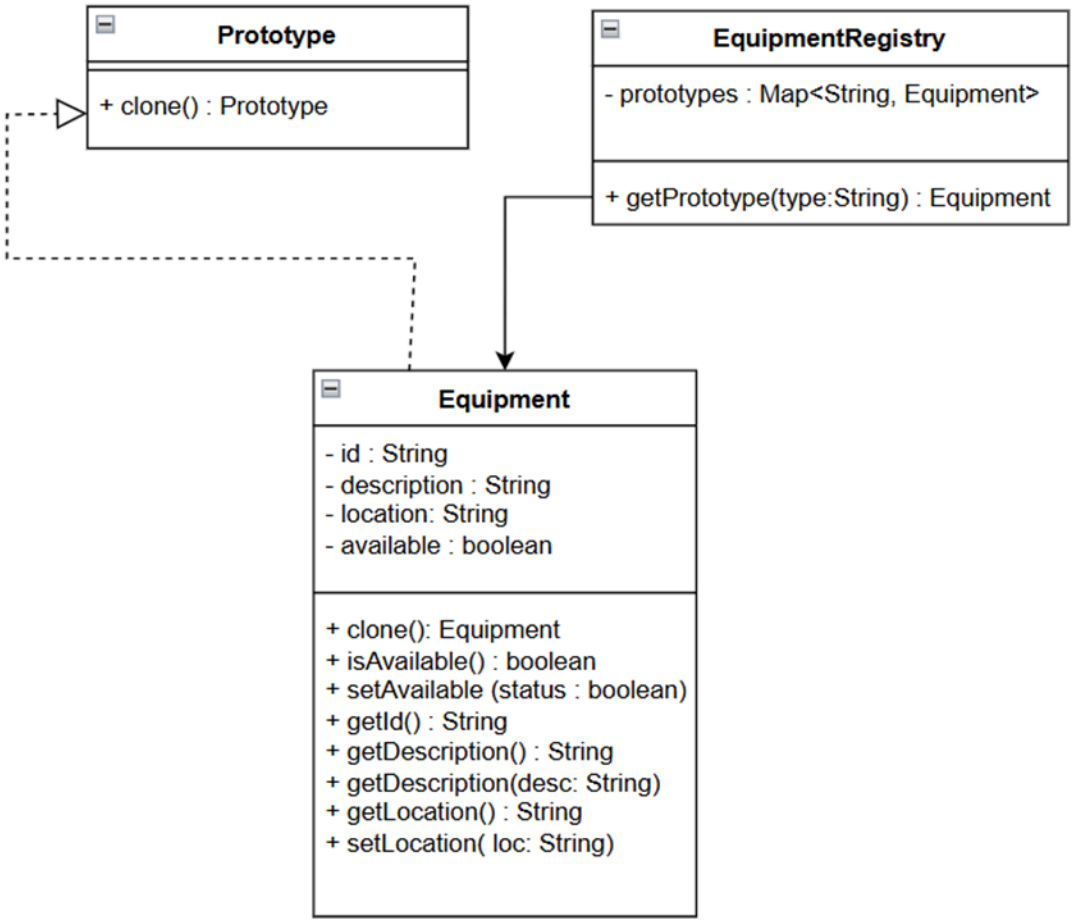
The Command pattern is used to encapsulate user actions triggered from the GUI into discrete command objects. Each command class (ReserveCommand, CancelCommand, ExtendCommand) implements the Command interface and defines an execute() method that delegates the action to the BookingFacade. By packaging the action, its parameters, and its target object together, the system decouples the GUI from the booking logic. This enables flexible features such as action logging, undo/redo, or queuing of commands. The pattern also keeps the GUI clean, as button clicks simply invoke execute() on the appropriate command object. This design aligns with the Command pattern’s goal of treating actions as first-class objects.

B. Class diagrams (contains six design patterns), you can use sub-diagrams to illustrate each of the six design patterns.

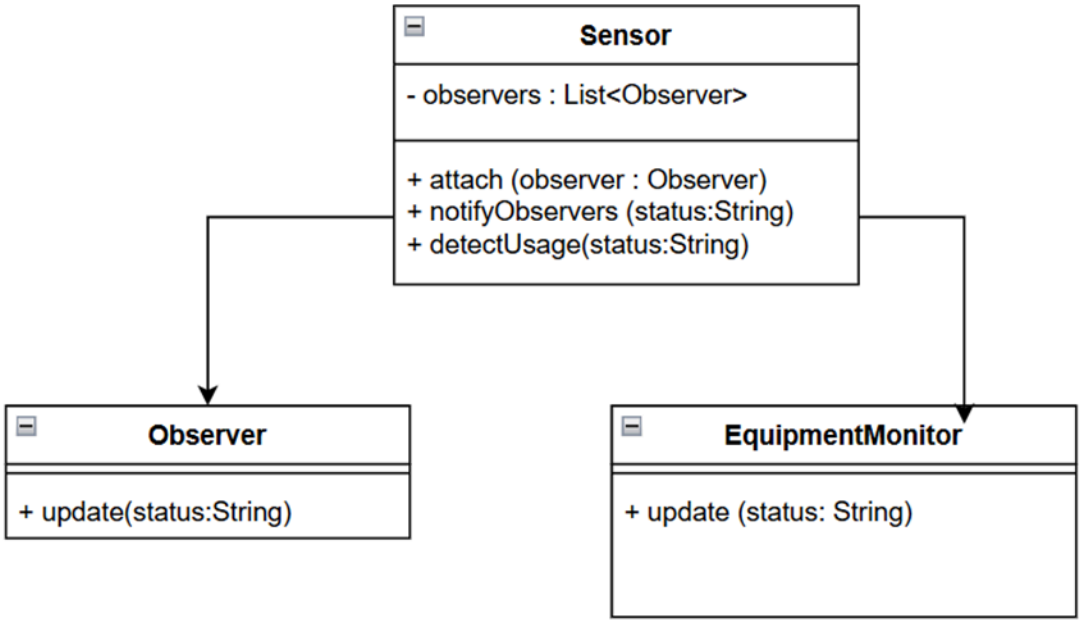
- Singleton Pattern - UserRegistry



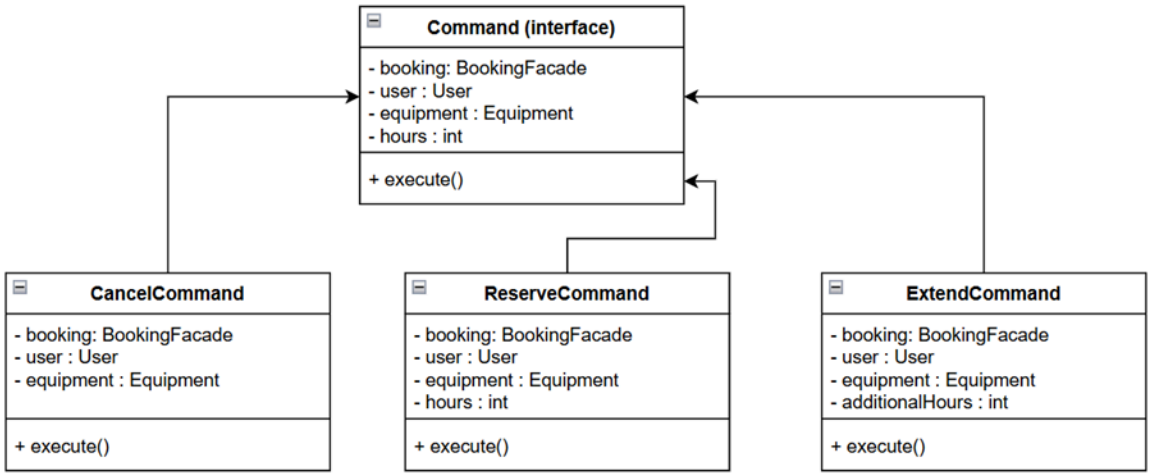
- Prototype Pattern - Equipment



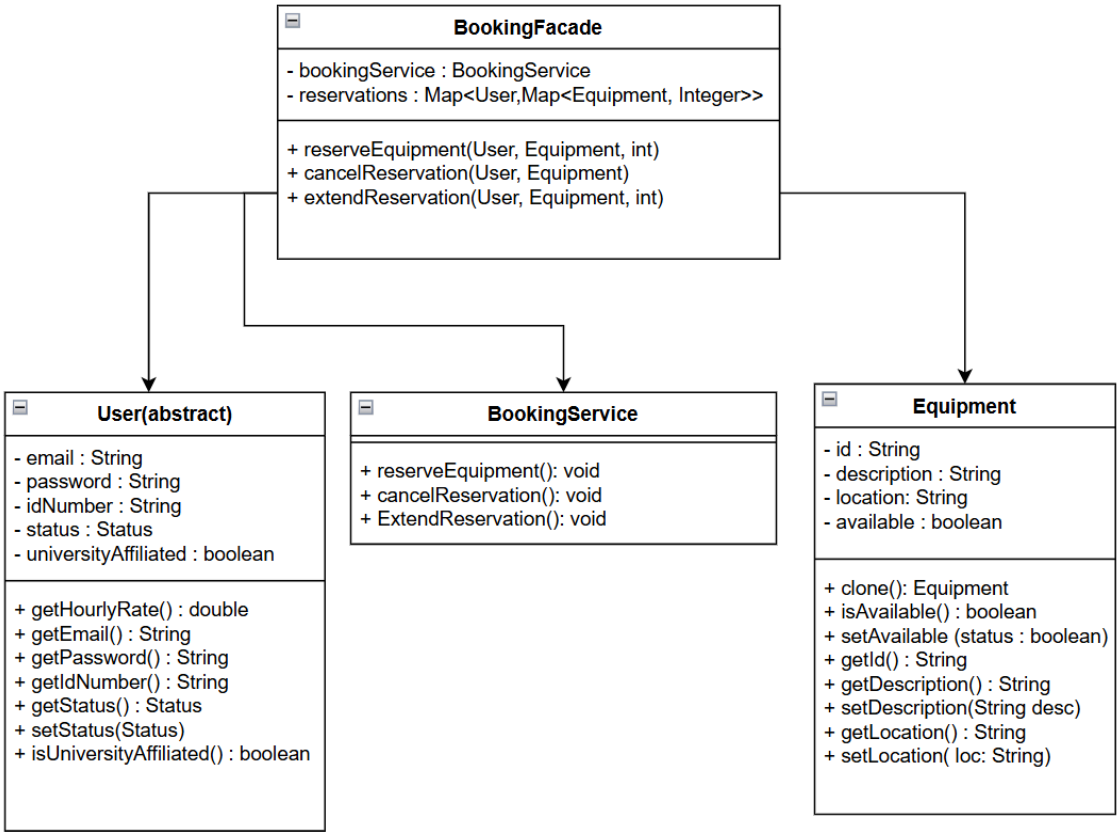
- Observer Pattern - Sensor, Observer, EquipmentMonitor



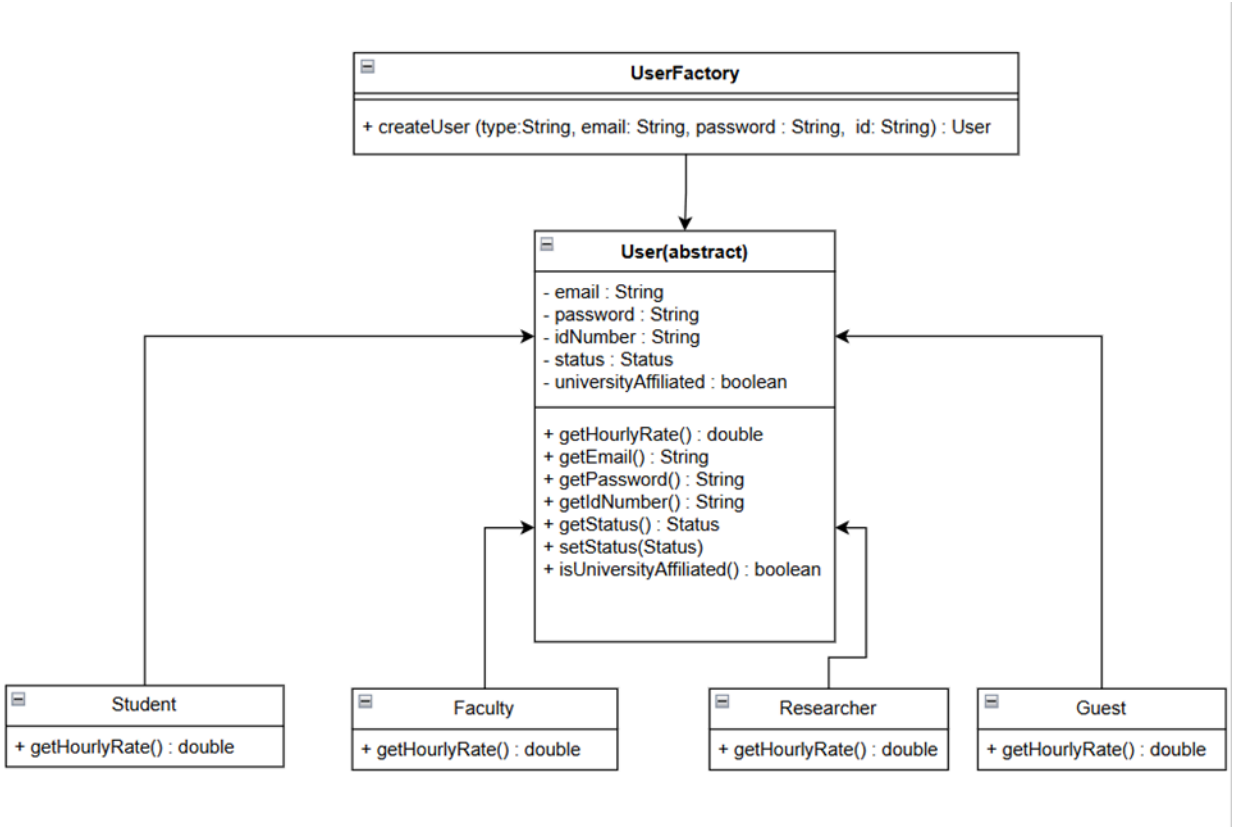
- Command Pattern - ReserveCommand, CancelCommand, ExtendCommand



- Facade Pattern - BookingFacade



- Factory Method Pattern - User Creation



C. Discussion about how each requirement can be achieved.

Requirement 1 - User Registration and Authentication:

Users register in the system using a unique email and password. The password has to meet the strong password requirements such as uppercase letters, lowercase letters, numbers, and symbols. This will all be handled by the PasswordValidator class. When a user registers, the system finds the user type and creates the correct object using the factory pattern. The UserFactory class is responsible for creating different subclasses of users, which are student, faculty, researcher, and guest. Each user object stores information such as email, password, and ID number. University affiliated users are first assigned the status of pending_approval using user.status until approval is granted by the head lab coordinator.

Requirement 2 - Lab Manager Account Creation:

Only the head lab coordinator can make the lab manager accounts. This is implemented through the head lab coordinator class. The coordinator checks pending users through the GUI and can approve or reject their registration. Once the user status is approved it is updated from pending_approval to approved. The lab coordinator also has access to create a lab manager account that can then later manage equipment and system operations.

Requirement 3 - Equipment Reservation and Pricing:

The users can reserve equipment using the systems booking interface. The reservation process is managed through the Facade pattern, which is implemented through the booking facade class. The system calculates reservation costs based on the user type. Each subclass of the user class has its own hourly rate, making sure that correct pricing is applied for students, faculty, researchers, and guests.

Requirement 4 - Deposit Requirements:

The system requires users to pay a deposit equal to one hour of the reservation fee before confirming a booking, This is managed within the reservation workflow by the booking facade. The deposit amount is from the users hourly rate and is integrated into the overall reservation process. This makes sure that all users follow the same deposit policy.

Requirement 5 - Equipment Usage Monitoring:

The system integrates with sensors to monitor equipment usage using the observer pattern. The sensor class acts as the subject while components like the equipment monitor act as the observers. When a sensor detects a change in equipment status it notifies all registered observers. These observers then update their internal state. This allows the system to respond to real time equipment usage changes while maintaining loose coupling between components.

Requirement 6 - Equipment Management:

Lab managers can manage equipment through the system interface. Equipment is represented by the equipment class which has attributes like ID, description, location, and availability. Managers can add new equipment, enable or disable existing equipment, and mark equipment as unavailable for maintenance. This is done by updating the availability status of equipment objects.

Requirement 7 - Equipment Identification:

Each equipment item is uniquely identified within the system. The equipment class has a unique ID, description, and lab location. This information allows users to view and select equipment correctly when making reservations, making sure clarity and preventing conflicts.

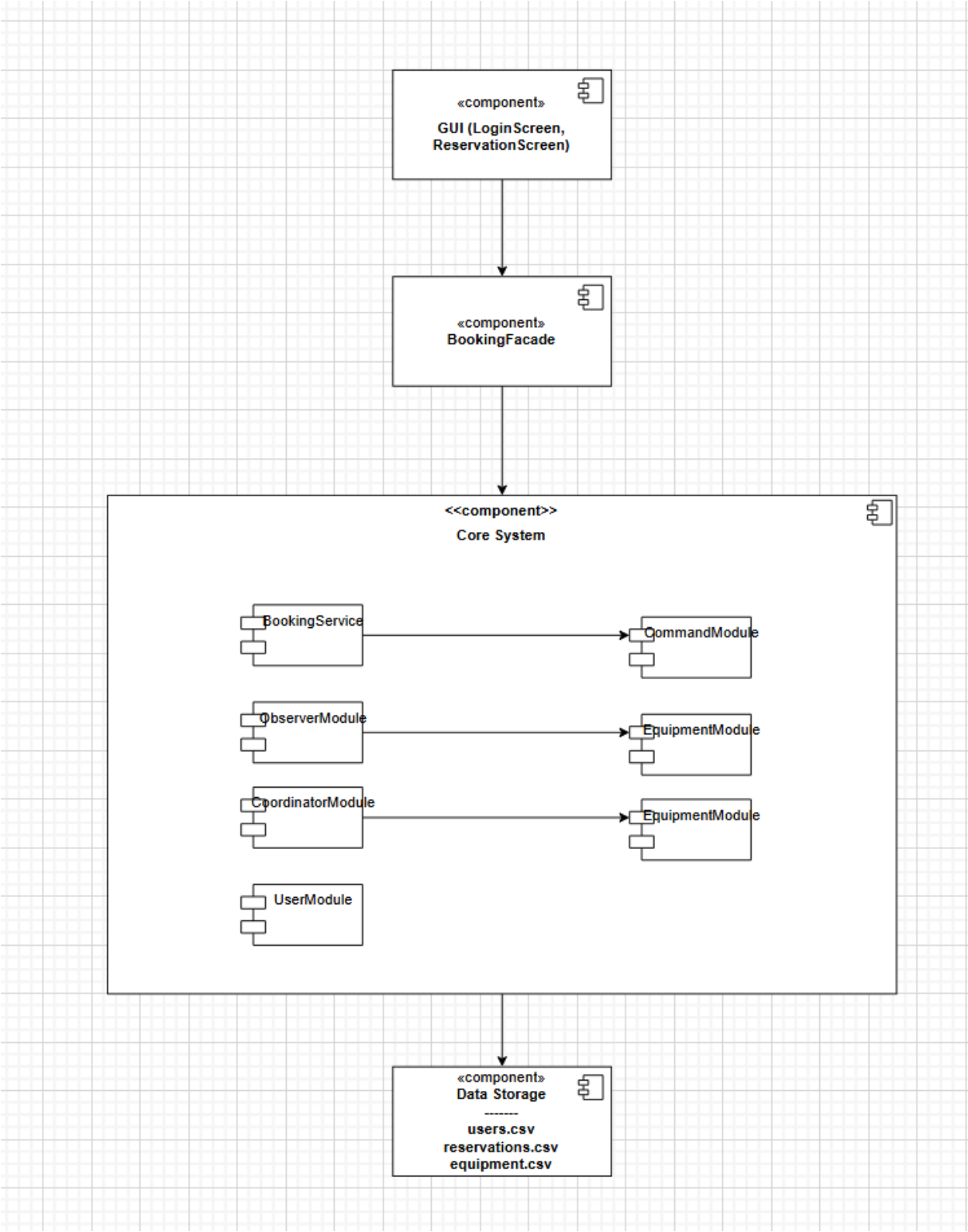
Requirement 8 - Reservation Modification and Cancellation:

Users are required to provide an identification number during registration which is stored in the user class. This makes sure that all users are verified. The system allows users to modify or cancel reservations before the booking start time. These are implemented using the command pattern, like the classes reserve command, cancel command and extend command. Each command is one reservation action such as reserve, cancel, or extend and then tells the booking system to carry it out.

Requirement 9 - Reservation Extension:

Users can extend their reservation if the equipment is still available after their current booking ends. This is handled by the booking facade. The system checks if the equipment is free and then updates the reservation time. Any extra cost if calculated using the users hourly rate.

D. Component diagram



E. Justification for component decomposition and interactions

The YorkU Lab Equipment Reservation Platform is decomposed into GUI, BookingFacade, Core System (modules), and Data Storage to achieve separation of concerns, modularity, and efficient coordination between subsystems. This architecture is guided by system requirements and reinforced using six design patterns: Factory Method, Prototype, Singleton, Observer, Facade, and Command. The decomposition ensures that each part of the system has a clearly defined responsibility while minimizing dependencies between components.

Justification for Component Decomposition

1. **GUI Component (LoginScreen, ReservationScreen)**

The GUI component handles all user interactions, including registration, login, and reservation operations.

Justification: Separating the GUI from the core system isolates presentation logic from business logic. This improves maintainability and allows the interface to evolve (e.g., transitioning to a mobile or web-based UI) without impacting internal functionality. The GUI does not directly interact with internal modules, reducing complexity and enforcing a clean architectural boundary.

2. **BookingFacade Component**

The BookingFacade acts as the single entry point between the GUI and the Core System.

Justification: Reservation processing requires coordination between multiple subsystems, including user validation, equipment availability, pricing, and booking management. The facade simplifies these interactions by providing high-level operations such as reserving, canceling, and extending reservations. This reduces coupling between the GUI and internal modules and ensures that complex workflows are handled consistently in one place.

Design Pattern: Facade Pattern

3. **Core System Component**

The Core System contains the main business logic and is divided into specialized modules to ensure high cohesion and clear responsibility separation.

- **UserModule**

Handles user registration, authentication, and role management.

Justification:

This module encapsulates all user-related functionality, making the system easier to extend with new user types. The use of the **Factory Method Pattern** allows different user objects (Student, Faculty, Researcher, Guest) to be created without exposing instantiation logic, improving flexibility and scalability (Requirement 1).

- **EquipmentModule**

Manages equipment data, availability, and configuration.

Justification: Equipment items often share similar attributes but differ in identifiers and location. The Prototype Pattern allows new equipment objects to be created by cloning existing templates, reducing redundancy and simplifying initialization (Requirement 7). This improves efficiency when managing large numbers of similar equipment.

- **BookingService**

Coordinates reservation workflows and enforces business rules.

Justification: Centralizing booking logic ensures consistency across all reservation operations, including creation, validation, and updates (Requirements 3, 8, and 9). This separation improves maintainability by isolating business rules from both the interface layer and execution mechanisms, reducing duplication and potential errors.

- **CommandModule**

Handles reservation actions such as reserve, cancel, and extend.

Justification: The Command Pattern encapsulates each booking action as an object, separating request invocation from execution. This improves extensibility, as new actions can be added without modifying existing code, and supports consistent handling of user operations (Requirements 8 and 9).

- **ObserverModule**

Manages real-time updates from equipment sensors.

Justification: The Observer Pattern enables the system to respond dynamically to changes in equipment status. Sensors act as subjects and notify observers when changes occur, allowing the system to update equipment availability in real time. This ensures responsiveness while maintaining loose coupling between hardware and software components (Requirement 5).

- **CoordinatorModule**

Handles administrative tasks such as approving users and creating lab manager accounts.

Justification: This module isolates privileged operations, ensuring proper access control and supporting Requirement 2. It prevents unauthorized access to administrative functionality and keeps security-related logic centralized.

- **ReservationManager**

Ensures a single instance manages all reservation data and operations.

Justification: The Singleton Pattern guarantees a single, consistent point of control for reservations. This prevents conflicting updates, duplicate bookings, and inconsistent system states, ensuring data integrity across all reservation processes.

4. **Data Storage Component**

Stores persistent system data such as users, reservations, and equipment records.

Justification: Separating data storage from business logic ensures that persistence concerns are isolated. This improves flexibility, allowing the system to transition from simple file storage to a database without affecting other components. It also ensures a centralized and consistent source of data.

Justification of Component Interactions

1. **GUI → BookingFacade**

The GUI communicates exclusively with the BookingFacade.

Justification: This enforces low coupling by preventing the GUI from directly depending on internal modules. It simplifies the interface and ensures that any changes in the core system do not impact the presentation layer.

2. **BookingFacade → Core System Modules**

The facade coordinates interactions with all core modules.

Justification: Since reservation workflows involve multiple subsystems, centralizing communication in the facade avoids duplication of coordination logic. It ensures that all operations follow a consistent process and reduces the risk of errors caused by scattered interactions.

3. **BookingService → CommandModule**

Booking actions are executed through command objects.

Justification: This interaction separates business logic from execution details, improving modularity and enabling the system to support additional operations without modifying existing workflows.

4. **ObserverModule → EquipmentModule**

Sensor updates trigger observer notifications, which update equipment status.

Justification: This enables real-time responsiveness while maintaining loose coupling. Equipment does not need direct knowledge of sensors, making the system more flexible and easier to maintain.

5. **Core System → Data Storage**

All modules interact with the Data Storage component for persistence.

Justification: This ensures a single source of truth for system data, improving consistency and reliability. Centralized data access also simplifies data management and future system upgrades.

Integration of Design Patterns

The system architecture integrates the six design patterns as follows:

- **Factory Method** → UserFactory (user creation)
- **Prototype** → EquipmentModule (equipment cloning)
- **Singleton** → UserRegistry (centralized control)
- **Observer** → ObserverModule (sensor updates)
- **Facade** → BookingFacade (system interface)

- **Command** → CommandModule (reservation actions)

Each pattern is applied within a specific component, ensuring that design decisions are structured, purposeful, and aligned with system requirements.

Architectural Benefits

This component decomposition and interaction design provides several key benefits:

- **Low Coupling:** Components interact through well-defined interfaces, minimizing dependencies
- **High Cohesion:** Each module focuses on a single responsibility
- **Scalability:** New features and components can be added with minimal impact
- **Maintainability:** Changes in one part of the system do not affect others
- **Flexibility:** Components such as the GUI or data storage can be replaced independently
- **Extensibility:** Design patterns allow the system to grow without major restructuring

Summary

The system is decomposed into well-defined components that reflect both functional requirements and sound design principles. The BookingFacade simplifies communication between the GUI and the Core System, while specialized modules encapsulate specific responsibilities. Interactions between components are carefully structured to reduce complexity and enforce consistency. By integrating six design patterns into the architecture, the system achieves a robust, scalable, and maintainable design that satisfies all requirements and supports future extension.

Task 2

The implementation of a runnable project
Github link: https://github.com/bubblebookmark/yorku-part2/tree/main

Task 3: Demo video (20mins)

Screen Recording 2026-03-22 at 7.56.30 PM.mov

Task 4

Provide a report describing how AI assistants (e.g., ChatGPT, Copilot, etc.) were used during the redesign and implementation of the project.

A. Role of AI in tasks 1 and 2

I used AI to confirm my understanding of the six required design patterns (Factory Method, Singleton, Prototype, Observer, Facade, Command). The assistants helped me break down each pattern into its essential components and understand how they typically appear in UML and code. When I drafted initial versions of the classes, I used AI to check whether the relationships (inheritance, associations, dependencies) aligned with the intended patterns. This was especially helpful for the Facade and Command patterns, where the boundaries between classes can be subtle. AI helped me refine method names, attribute visibility, and diagram labels so that everything matched the actual Java code. This ensured that the UML diagrams were not only correct but also professional and consistent. When I wasn't sure how to test certain parts of the system (e.g., cloning in the Prototype pattern or observer notifications), AI provided small test snippets and suggestions that helped me verify the behaviour.

B. Arguments and resolution

There were several moments where the AI's suggestions conflicted with my own interpretation or with the assignment requirements. These disagreements actually helped me refine my understanding.

Prototype Pattern Disagreement: AI initially suggested adding a Prototype interface and an EquipmentRegistry, which are common in textbook examples. However, my actual implementation only required Equipment implements Cloneable.

Resolution: I chose to keep my implementation simple and adjust the diagram to match my real code. This reinforced the idea that the assignment values correctness over unnecessary complexity.

Facade Pattern Method Signatures: AI pointed out that my UML diagram for BookingService did not match the actual method names in my code. The diagram showed methods like reserveEquipment() and cancelReservation(), while the real class only had reserve(), cancel(), and extend().

Resolution: I updated the diagram to match the code exactly. This helped me understand the importance of consistency between UML and implementation.

Command Pattern Attributes: At one point, AI suggested including attributes inside the Command interface. However, interfaces in Java cannot contain instance fields.

Resolution: I kept the attributes only in the concrete command classes. This ensured the design followed Java's rules and the Command pattern correctly.

C. Verification of AI responses

To ensure that AI suggestions were accurate and aligned with the course expectations, I used several verification strategies: Whenever AI suggested a structural change, I compared it with the examples provided in class. If something didn't match the course's interpretation of a pattern, I adjusted accordingly. For patterns like Prototype, Observer, and Command, I wrote small test drivers to confirm that the behaviour matched the UML diagrams and the intended pattern roles.I made sure that UML diagrams matched the Java code, Method names were identical, Associations in diagrams reflected actual object interactions, and No pattern was over-engineered beyond what the assignment required. I treated AI suggestions as feedback, not instructions. If something didn't make sense or didn't match the assignment, I revised it myself.

D. Conclusion

AI assistants played a helpful but controlled role in the redesign and implementation of the project. They were used mainly for clarification, validation, and refinement and not for generating the entire solution. All final design decisions, diagrams, and code were created and verified by me to ensure they met the course requirements and accurately implemented the six design patterns.